

Documentation technique

Application « ScorFlow » — architecture, modèle de données et déploiement

Projet	ScorFlow — Renouvellement de crédits
Version	1.0
Date	Juillet 2026
Public	Équipe technique DSI
Confidentialité	Interne

1. Architecture générale

ScorFlow repose sur deux services distincts déployés sur le même serveur interne. Une interface web Django assure l'authentification, l'affichage et les écrans de décision ; elle ne parle jamais directement à la base métier. Un service FastAPI expose une API interne et concentre tous les accès à la base Oracle (lecture des propositions, appel des procédures de décision et de génération). Les deux services sont exécutés par Gunicorn et pilotés par systemd.

Composant	Technologie
Interface web	Django 5 (Python 3), Gunicorn, service systemd dédié
API métier	FastAPI, Gunicorn/Uvicorn, service systemd dédié
Base métier	Oracle (pilote python-oracledb, pool de connexions)
Base applicative	SQLite (comptes, sessions et groupes Django uniquement)
Interface	HTML, Bootstrap 5, Chart.js (graphiques), Flatpickr (calendrier)
Système hôte	Ubuntu Server

2. Modèle de données

Les données métier résident intégralement dans Oracle. L'application ne duplique aucune donnée de crédit ; elle lit une vue de synthèse et délègue les écritures à un paquetage PL/SQL.

Objet Oracle	Rôle et champs principaux
V_RENOUVELLEMENT	Vue de lecture des propositions : CODE_REGION, ID_PROPOSITION, NO_PRET_SCORE, MT_PRET_ORIGINAL, SOLDE_A_RACHETER (encours), STATUT_PROPOSITION, MATRICULE_CLIENT, NOM_COMPLET, CODE_BUREAU, LIBELLE_BUREAU, SCORE_TOTAL, MT_PROPOSE, MT_ACCORDE, D_PREM_ECH, PRET_GENERE.
PROPOSITION_RENOUVELLEMENT	Table porteuse des propositions et des décisions.
REGION	Référentiel des agences (CODE_REGION, LIB_REGION).
EVUTI	Référentiel des utilisateurs habilités (CUTI, CODE_REGION, CODE_SERV, DMMP).
pkg_renouvellement	Paquetage PL/SQL : enregistrer_decision (décision et contrôles métier), generer_tous_prets_comite (génération des prêts approuvés).

Côté Django, seule la base SQLite locale est utilisée, pour les comptes, les sessions et les groupes. Le rôle « reporter » est matérialisé par un groupe Django du même nom.

3. Règles de visibilité

- Membre du comité : les propositions sont filtrées par CODE_REGION, déterminé à partir de son matricule dans EVUTI (service 004, activité de moins de 90 jours). Un utilisateur absent d'EVUTI est redirigé vers une page « non autorisé ».
- Reporter : membre du groupe Django « reporter », il est redirigé dès la connexion vers le reporting global (toutes agences), en lecture seule.

- Le contrôle est systématiquement effectué côté serveur, à chaque requête.

4. Sécurité

- Authentification par matricule et mot de passe ; création des comptes réservée à l'administrateur.
- Configuration sensible (clé secrète, identifiants Oracle, URL de l'API) hors du code, dans des fichiers d'environnement non versionnés.
- Le navigateur ne dialogue qu'avec Django (même origine) : aucune adresse ni port de l'API n'est exposé côté client.
- Protection CSRF, en-têtes de sécurité, mode DEBUG à désactiver en production.
- Les règles métier sensibles (plafond du montant accordé, prêt déjà généré) sont appliquées par Oracle ; les messages d'erreur applicatifs sont nettoyés avant affichage à l'utilisateur.

5. Traçabilité

Chaque décision est enregistrée par la procédure enregistrer_decision avec le matricule de son auteur, la date, le statut retenu, le montant accordé et le commentaire. La génération des prêts est réalisée par une procédure unique, à l'échelle de l'agence, ce qui garantit l'homogénéité du traitement.

6. Déploiement

Les deux services sont installés dans un environnement isolé (dossier projet, environnement virtuel Python et services systemd dédiés). La mise à jour consiste à récupérer le code puis à redémarrer le ou les services concernés.

Élément	Valeur
Adresse d'accès (réseau interne)	http://192.168.0.122:4884/
Service interface web	django.service (Gunicorn, port 4884)
Service API métier	fastapi.service (Gunicorn/Uvicorn, port 8008)
Environnement Python	Environnement virtuel dédié au projet
Configuration	URL de l'API via la variable FASTAPI_URL ; identifiants Oracle dans le fichier d'environnement du back-end
Mise à jour	git pull → redémarrage du service modifié (interface et/ou API)

Les fichiers d'environnement (.env) et les environnements virtuels ne sont pas versionnés : un git pull ne les écrase jamais.